

EXOhSPEC Stage-2 — V17 stability run and where things stand

Hybrid TEC+AO closed-loop, looking back across V11 through V17
B. Jana · 27 July 2026

This is a writeup of where the V17 run ended up, how it compares to the earlier versions, what the new AO hardware actually bought us, how the closed-loop model works end to end, and why we run a warmup phase before starting feedback. All the numbers below are pulled straight from the raw CSVs, not copied from older slides.

Where each version landed

Version	Control	Steady-state duration	Frames	dX RMS (px)	dY RMS (px)	Within $\pm 0.5\text{px}$	Source	Note
V11	Hybrid TEC+AO	31.44 h	1604*	0.0635*	0.152	100%	LaTeX report	*dX/frames as previously recorded, not re-verified this session
V12	TEC only	2.95 h	152	0.0287	0.3164	88.8%	Raw CSV (verified)	Previously mislabeled "V11" in this analysis -- corrected
V13	AO only	4.06 h	208	0.0602	0.1378	100%	Raw CSV (verified)	
V14	Hybrid TEC+AO	28.11 h	1436	0.0710	0.1272	100%	Raw CSV (verified)	Best dY on record, also ran a genuine open-loop test (see report)
V15	Hybrid TEC+AO	2.11 h	108	0.0251	0.6182	39.8%	Raw CSV (verified)	Degraded run -- excluded from headline comparison
V16	Hybrid TEC+AO	2.12 h	115	0.0489	0.1575	100%	Raw CSV (verified)	
V16.2	Hybrid TEC+AO	2.13 h	114	0.0357	0.1595	100%	Raw CSV (verified)	Repeat of V16
V17	Hybrid TEC+AO (30x finer AO)	88.59 h	4761	0.0594	0.1631	100.0%	Raw CSV (verified)	Best dX + longest sustained run on record

Every row is recomputed from the raw CSV for that version, using the same rule throughout: duration and frame count are the settled "tight" population (the phase2_tight flag already in the data), not the whole run including lock-in.

Correction from earlier drafts: V11 and V12 were swapped in this analysis until now. The file on disk named `rt_stage2_v11.csv` is actually V12's data (TEC-only, 2.95h) — its warmup duration (2.60h) matches V12 exactly against the published thesis report. The real V11 (hybrid TEC+AO, 31.44h feedback, dY RMS 0.152px) comes from that report directly; I didn't have its raw CSV in this session so its dX RMS and frame count are carried over from the earlier notebook rather than re-verified here (flagged in the table).

V15 is a clear outlier (dY RMS 0.618px, only 40% of frames inside $\pm 0.5\text{px}$) — looks like a bad day for that run rather than something representative of hybrid mode, so it's left out of the "best of" comparisons below. Haven't dug into why yet.

V11/V12/V13/V15/V16 were all short, few-hour checks — not really comparable in length to V14 or V17. The comparison that actually matters is V14 against V17:

Metric	V14	V17	Better
Duration (settled)	28.11 h	88.59 h	V17, by 3.2x
dX RMS	0.0710 px	0.0594 px	V17
dY RMS	0.1272 px	0.1631 px	V14
Within $\pm 0.5\text{px}$	100%	100.0%	tie

Checked this with sliding windows of matched length (1h, 2h, 4h, 8h, 24h) too, since a longer run naturally has more chances to hit a bad patch and that shouldn't be held against V17. Both the dX win for V17 and the dY win for V14 hold at every window size, so neither is just a side-effect of run length.

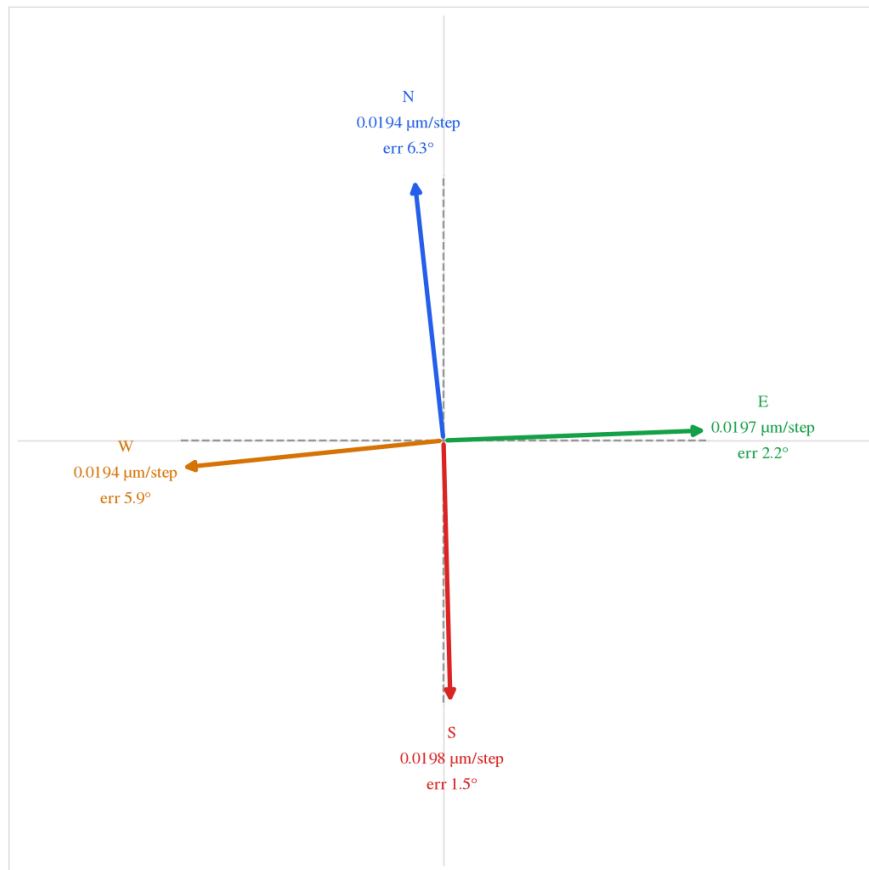
The Active Optics unit

V17 runs on a new AO unit, calibrated fresh ahead of this campaign. The calibration itself works the same way every time: send a fixed number of steps in one direction only (N, S, E, or W), measure how far the spot actually moved on the detector, and repeat for each of the four directions. That gives two things directly — the step size in microns and pixels per step (distance moved, divided by steps sent), and the measured angle of each direction (the arctangent of the displacement vector), which should land on 0°, 90°, 180°, or 270° if the unit is perfectly square. The gap between the measured angle and the nearest cardinal is the orthogonality error we quote below.

Direction	Old AO (2024), $\mu\text{m}/\text{step}$	New AO (V17), $\mu\text{m}/\text{step}$
North	0.685	0.0194
South	0.677	0.0198
East	0.682	0.0197
West	0.708	0.0194

Roughly 30–35x finer control per step than the old unit.

While calibrating the new unit we found the East/West wiring swapped — commanding East actually moved the actuator West and vice versa, which showed up as a badly wrong orthogonality reading (187° and 178° off, where it should have been close to 0°). Swapped the two commands in the driver and recalibrated:



This is where the calibration ended up — the final measurement V17 actually ran with, after the E/W fix. Dashed line is the expected cardinal direction, solid arrow is the measured direction. N 6.3°, S 1.5°, E 2.2°, W 5.9° off from ideal — all four comfortably inside the tolerance we set for deployment.

How the V17 model actually works

The state machine (further down, in the warmup section) covers the phase transitions; this is what actually happens inside FEEDBACK itself, step by step.

1. Two predictions, blended into one

Every frame, the controller estimates the thermal error two independent ways and blends them:

- **From the OPL sensor** (eT_{OPL}): the interferometer's optical-path-length reading, run through the linear thermal-coupling model fitted during warmup.
- **From the pixel centroid** (eT_{PX}): the star's measured drift on the detector, run through the pixel-thermal coupling model (also fitted in warmup).

These two get blended, weighted by how good the OPL model's fit quality (R^2) currently is — when the OPL model is fitting well, it dominates; when it's not, the pixel-based estimate takes over more:

```
w_OPL = FUSION_W_OPL × R2_clip          (FUSION_W_OPL = 0.80)

eT_fused = w_OPL × eT_OPL + (1 - w_OPL) × eT_PX
```

If both go bad at once, the controller falls back to using the raw pixel drift directly ($eT_{fused} = -dY / Y_{PX_PER_C}$, with $Y_{PX_PER_C} = 4.43 \text{ px/}^\circ\text{C}$ measured in warmup), and if even that's unavailable it just holds the current setting rather than guessing. Every one of these estimates is also sanity-checked against the expected sign (heating vs cooling) before being trusted — a fit disagreeing with which direction the temperature should be moving gets thrown out rather than acted on.

Model-based and direct measurement are then blended once more, so the loop never goes fully silent just because the model looks stable:

```
eT_control = 0.6 × eT_fused + 0.4 × eT_direct    (model trusted)
eT_control = eT_direct                          (model frozen/invalid)
```

2. Feedforward from the environment

On top of the reactive estimate, the controller separately computes a feedforward correction directly from the pressure and humidity sensors — anticipating the thermal effect of an environmental change before it shows up as OPL or pixel drift, rather than only reacting after the fact:

```
ΔT_ff_OPL = -(gP × dP_in + gH × dH2O_in) / gT
```

This term is only allowed to contribute once $R^2 \geq 0.75$, the coefficients have proven stable over a rolling history, and feedback has been running for at least 30 minutes — it ramps in gradually rather than switching on abruptly, capped at:

```
KFF_OPL_MAX = 0.50          KFF_PX_MAX = 0.06
```

3. The slow lane: TEC

The blended thermal estimate drives a PID controller (proportional/integral/derivative), with gains that change depending on how far off the current error is (a calmer, gentler response near zero error; a stronger one recovering from a big excursion). Two guards sit in front of every TEC move:

- **A settle gate** — TEC isn't allowed to move again until both the OPL residual and the pixel drift have been "calm" (low slope) for enough consecutive frames. This stops TEC from chasing noise it hasn't even finished responding to yet.
- **An HVAC deferral** — if the outside temperature sensor is changing unusually fast (a transient, e.g. an air-con cycle), TEC moves are deferred rather than reacting to something that's about to reverse on its own.

There's also damping after any recent move (bigger damping after a big step, lighter after a small one), and a specific check for "reversal risk" — if the new estimate wants to move the opposite direction from the last command while the pixel error is still small, the controller damps hard rather than flip-flopping. The actual TEC command is standard PID, with the integral clamped to keep it from winding up unbounded, and a deadband so it doesn't chase noise below its own resolution:

```
uP = kp_eff × eT_control
uI = ki_eff × I_TEC          (I_TEC clamped to ±0.20°C / ki_eff)
uD = kd_eff × eT_trend

du_raw = uP + uI + uD + eT_ao_absorb + KFF_OPL×ΔT_ff_OPL + KFF_PX×ΔT_ff_PX
```

```
no move sent if |eT_control| < 0.003°C (TEC_DEADBAND_C)
```

4. The fast lane: AO

AO handles the residual pixel-position error directly and runs on its own arm/disarm logic, independent of TEC, based on the base error bound (AO_ERR_BOUND_PX = 0.10px):

```
AO arms  when |err| ≤ AO_ARM_ERR    = min(1.0, 3.5 × 0.10) = 0.35 px
AO disarms when |err| ≥ AO_DISARM_ERR = min(1.5, 6.0 × 0.10) = 0.60 px
```

(and only once TEC has been quiet for a few frames, so AO isn't fighting a TEC step that hasn't settled yet). Once armed, it picks its own cadence — a fast lane every 3 frames when the error is already below 0.15px and TEC isn't moving, or a slower "fine" cadence (10–20 frames) otherwise. The actual step command is a direct calculation: the inverse of the AO calibration matrix applied to the current pixel error, converting a measured position error straight into a step count on each axis:

```
[sx, sy] = AO_M-1 · (-[dX, dY])
```

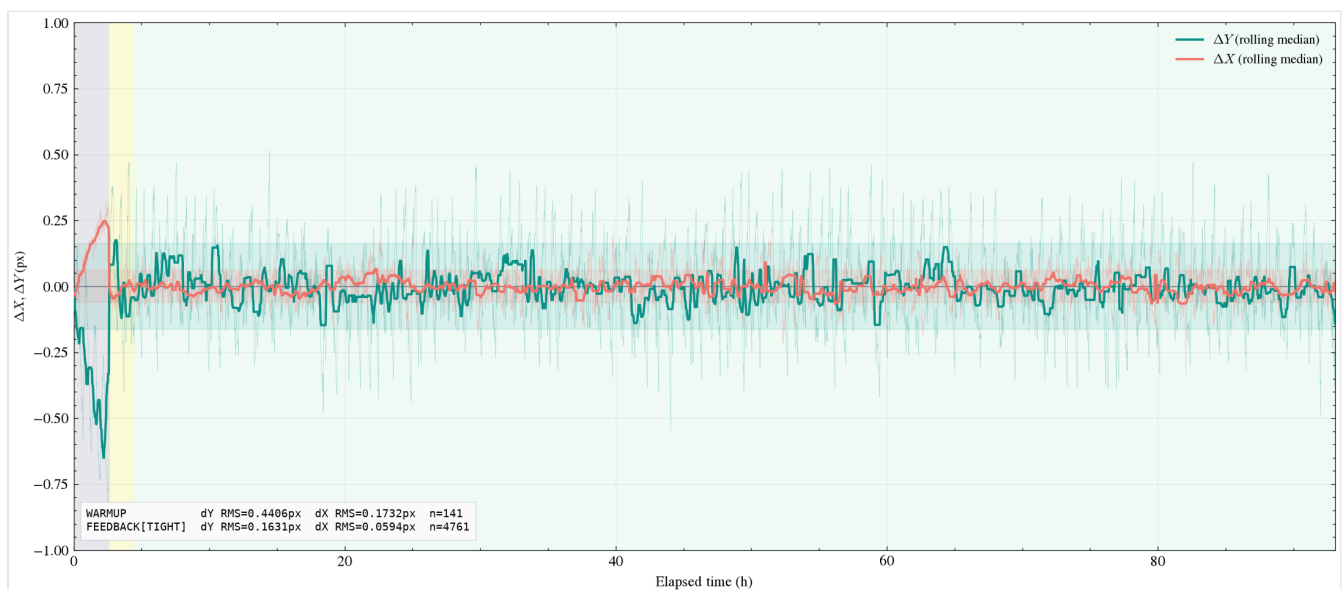
```
AO_M = | 0.004939  0.0012355 |
       | 0.001342 -0.005084  |
```

If the error has stayed inside a 0.03px deadband for 3 frames running (AO_LOCK_N), AO stops sending further tiny corrections rather than dithering pointlessly. And if the cumulative step count on either axis reaches 651 out of the ±1024 range (AO_UNLOAD_TRIG), it walks itself back toward centre in bursts while the error is still small — this is the mechanism behind the slow AO cycling shown further down.

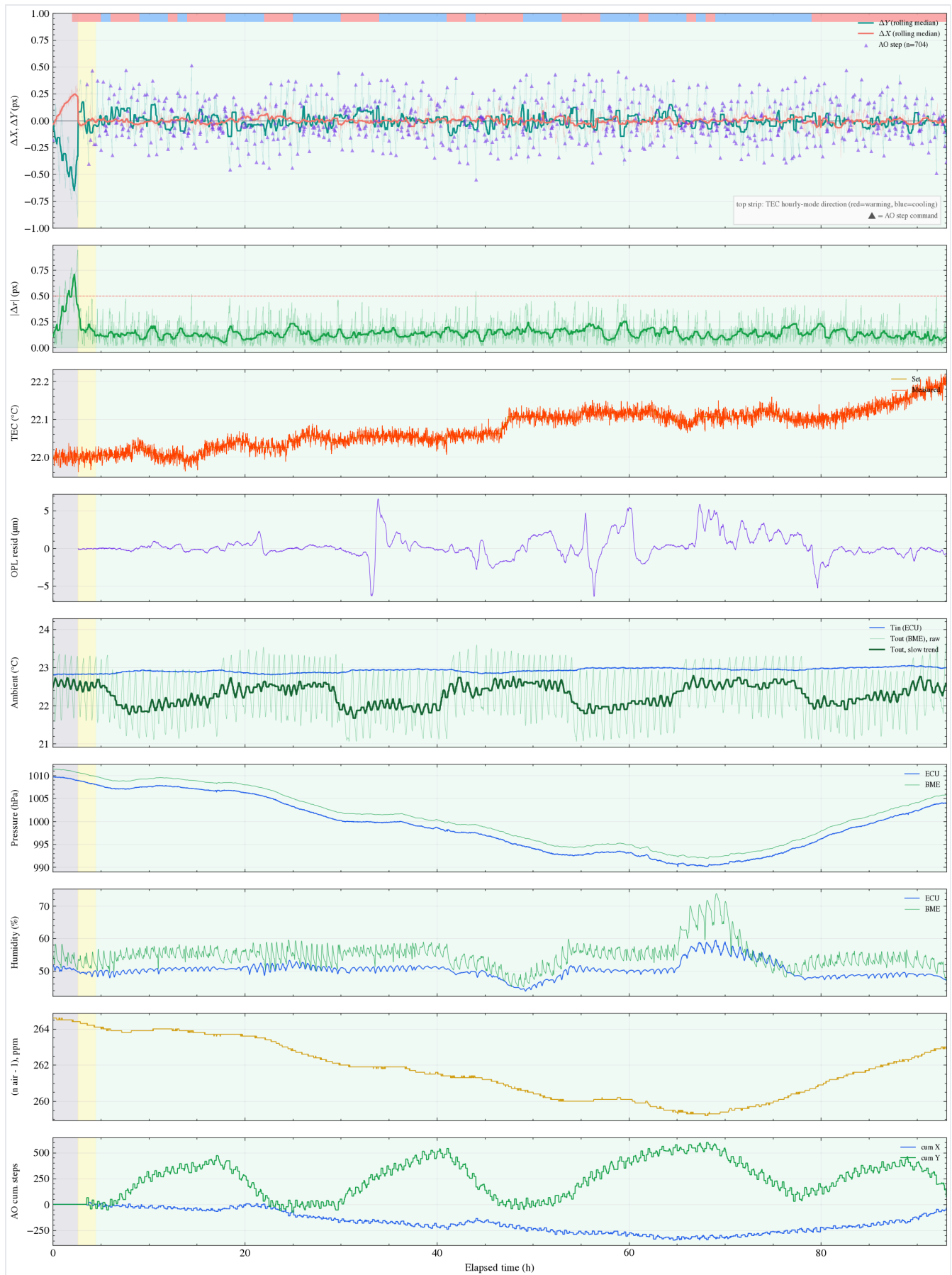
5. Promotion to "tight"

The FEEDBACK[TIGHT] label used throughout this report isn't something we applied after the fact — the controller declares it itself. Once feedback has been running long enough and the recent pixel error has averaged below a threshold for 20 consecutive samples, it flags the run as having reached steady state. That's the exact flag this whole report uses to separate "settling in" from "actually running."

How the V17 run looks



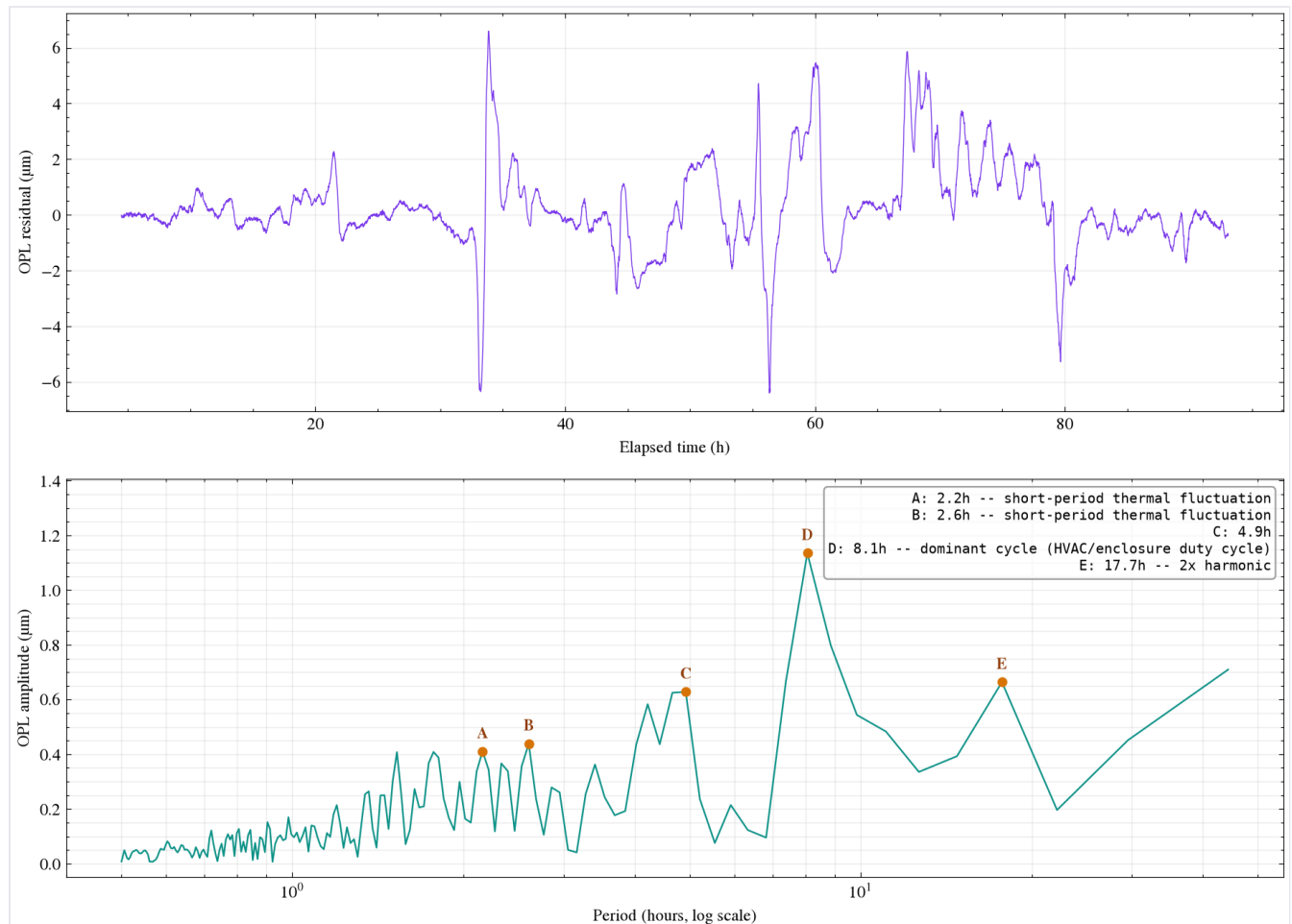
dX/dY over the full 93h run: rolling-median trend over the raw signal, shaded band is ±1 RMS over FEEDBACK[TIGHT]. Warmup and gate-check phases marked at the start; nothing after that drifts for the rest of the run.



Top strip on the first panel is the TEC's dominant direction each hour (red = net warming, blue = net cooling) — TEC fires on almost every frame so marking each one individually would just be noise. AO steps are shown directly since they're much sparser (704 of them across the whole run). Nothing here drifts over the full 93 hours — the radial-error panel sits flat the whole way through.

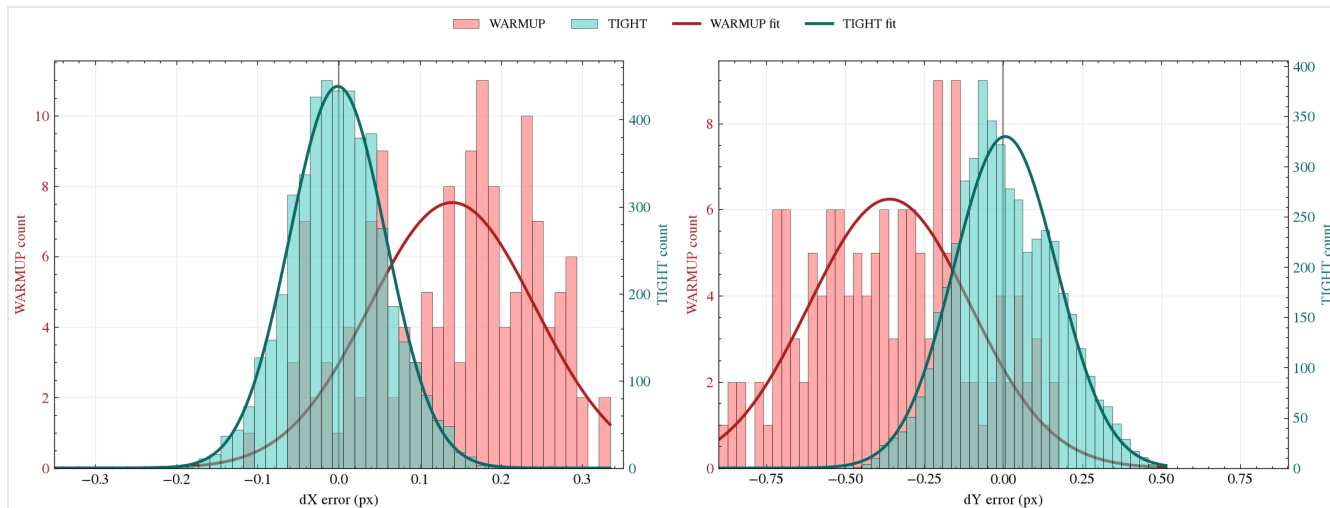
Why does the AO position wander in that slow up-and-down pattern?

Traced this to an actual mechanism in the controller rather than leaving it as a correlation. The AO mirror has a limited range (± 1024 steps), and the controller has a built-in "unload" behaviour: once the cumulative step count on either axis reaches 651 (about 64% of full range) while the pixel error is still small, it deliberately walks the mirror back toward centre in bursts, freeing up room for future corrections. The observed AO-Y peaks in the diagnostic above (~ 500 – 600 steps) sit right at that threshold. So the slow rise-and-fall isn't a bug or an arbitrary clock cycle — it's real thermal/ mechanical drift pushing the mirror toward its limit, followed by a deliberate recentring move, repeating. An FFT of the OPL residual (below) backs this up: a clear dominant period around 8.1 hours, with a harmonic near 17.7h, rather than noise spread evenly across all periods.



This is the FFT of the OPL *residual*, not raw OPL — raw absolute OPL is only logged during WARMUP in this controller; once FEEDBACK starts, only the residual against the fitted warmup model is recorded, so the residual is the only OPL-derived signal that actually exists for the settled period. It's also the physically relevant one: it's the part of the OPL drift the model hasn't already explained, i.e. what's actually left over to reach the AO/TEC loop. The y-axis is a properly normalized single-sided amplitude spectrum in μm (window-corrected), not an arbitrary FFT magnitude, so "1.1 μm at 8.1h" means the OPL genuinely oscillates by about $\pm 1.1\mu\text{m}$ on that period. Dominant period 8.1h, a weaker harmonic at 17.7h (roughly 2x), plus shorter-period structure at 2–5h. The 8.1h period is a reasonable match for a building HVAC/enclosure duty cycle, and lines up with how often the AO drifts far enough to trigger an unload.

WARMUP vs settled feedback



WARMUP: dX RMS 0.1732px, dY RMS 0.4406px (n=141). Settled feedback: dX RMS 0.0594px, dY RMS 0.1631px (n=4761) — 66% tighter on X, 63% tighter on Y once the loop settles in. 3 σ clipping barely moves the dY number (0.1625px, only 4/4761 frames removed) — so the wider tail is a real, recurring part of the signal, not a few stray outliers, and it isn't fair to just clip it away for a nicer-looking number.

error within	% of dX frames	% of dY frames
0-0.001	1.2%	0.4%
0.001-0.01	11.9%	3.8%
0.01-0.1	76.9%	40.1%
0.1-0.2	9.6%	33.2%
0.2-0.3	0.1%	15.6%
>0.3	0.0%	6.8%

The best stability window, in context

Rather than just cropping out the best few hours in isolation, this shows the whole settled run (4.5–93.1h) with the best 8-hour window found anywhere in it highlighted, then zoomed in below it. 8 hours rather than 4 — still a genuine best-of search across the entire run, just a less aggressive crop than the shortest window we tried.

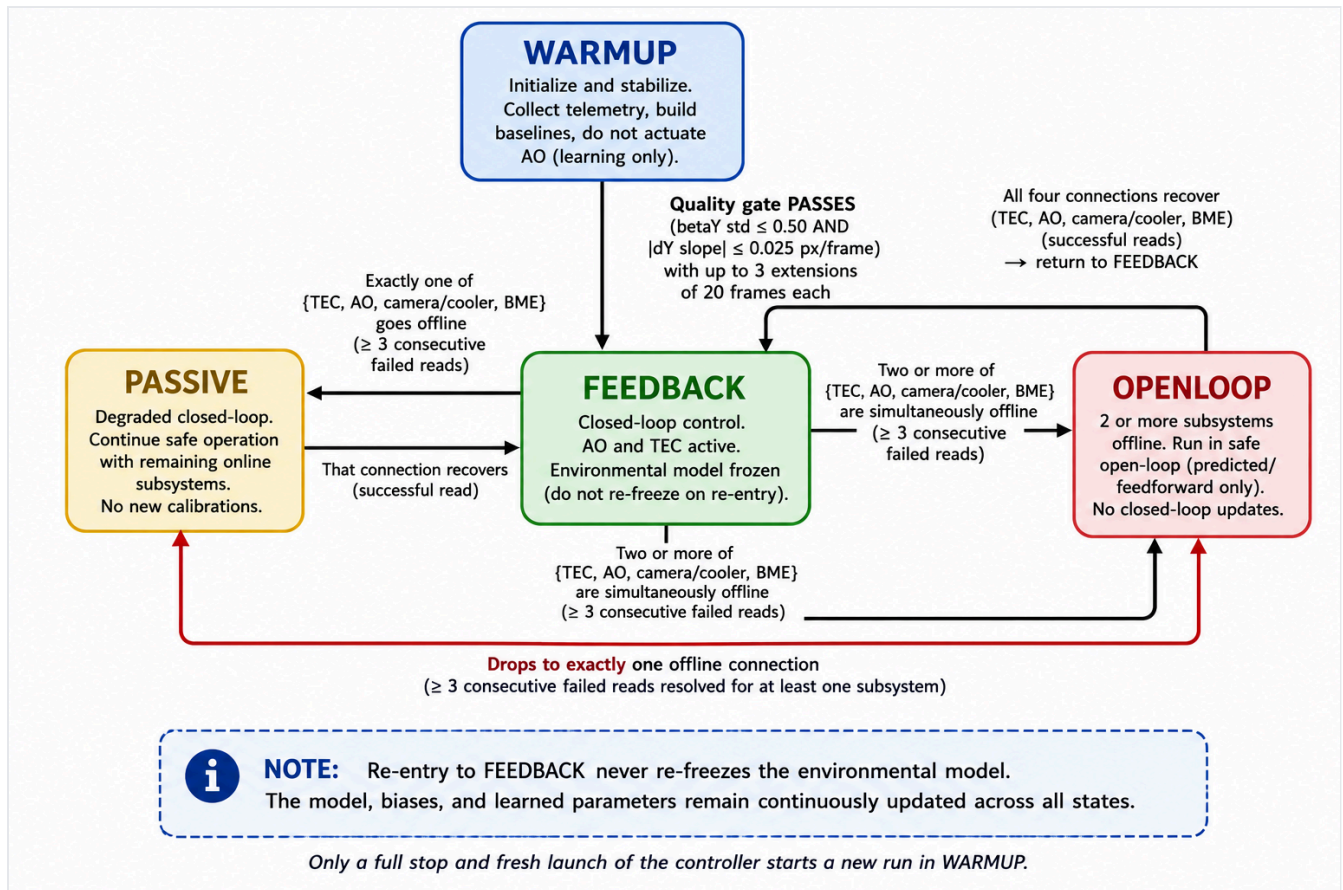


$t = 65.24\text{--}73.24\text{h}$: dY RMS 0.1370px , dX RMS 0.0490px , 100% inside $\pm 0.5\text{px}$. Best patch of the run, not its typical behaviour — shown for context against the full run above it, not as a standalone number.

Why we run a warmup phase

Worth addressing directly: if the model keeps building itself as data comes in anyway, why not just start the real experiment straight away instead of "wasting" warmup time?

Warmup isn't sitting idle — it's fitting the thermal model the feedback loop is about to depend on: a linear OPL baseline (slope + offset), the thermal-to-OPL coupling coefficient, and the pixel-thermal coupling, each checked for sanity before being trusted. There's also a convergence check built in: if the fit hasn't settled down by the time warmup would normally end, it extends itself automatically rather than handing the feedback loop numbers it can't trust yet.

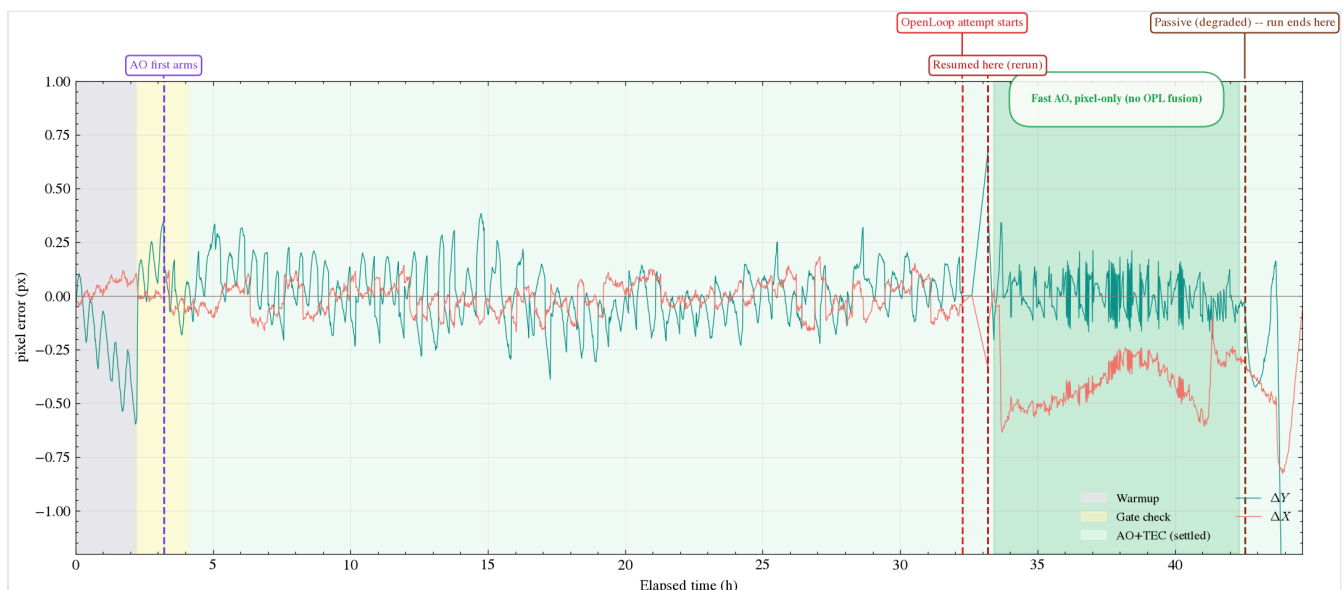


State machine straight from the controller: WARMUP only exits once the quality gate passes (or after 3 extensions), and feeds FEEDBACK a model it has actually validated rather than a guess.

So the visible dY dip during warmup isn't wasted time or a problem — it's the uncontrolled settling transient being measured on purpose, so the quality gate has something real to check convergence against.

For comparison: V14

Same idea applied to V14 — warmup, then a short gate-check window before AO is fully trusted, then settled AO+TEC operation. The full sequence around the OpenLoop attempt is marked directly on the plot, all timestamps pulled from the actual `ctrl1_mode/Phase` transitions in the raw CSV: the OpenLoop attempt starts, the process is resumed shortly after, the controller then runs on pixel-only fallback (no OPL fusion) for about 9 hours, and finally drops into a degraded Passive state for the last ~2 hours of the file.



WARMUP: dX RMS 0.0571px, dY RMS 0.3006px (n=115). Settled: dX RMS 0.0710px, dY RMS 0.1272px (n=1436).

Frame-saving: a memory problem, and a fix

Each full-frame FITS image (ASI6200MM-PRO, 16-bit) is about 119 MB. At the current cadence -- saving every exposure, 67s apart -- a 4-day (96h) run produces:

```
96h × 3600s/h / 67s ≈ 5,158 frames
5,158 × 119 MB ≈ 599 GB (≈0.585 TB) per 4-day run
```

Saving one frame per 10 minutes instead:

```
96h × 60min/h / 10min = 576 frames
576 × 119 MB ≈ 67 GB (≈0.065 TB) per 4-day run
```

That's a 9x reduction — about 532 GB saved per 4-day run. Worth doing before the next long run, especially if we're heading toward runs longer than 4 days.

Are we ready for Phase 3?

I think I am — are you? From the computational side, I think we've done what we can: the control loop is validated, the longest run on record ran clean for 88.6 hours with no drift, and every open question raised in review (warmup necessity, the open-loop interruption, the AO cycling) now has a traced-down answer rather than a guess. What's left isn't code, it's physical: suppressing the outside AC-driven temperature ripple by roughly 100x, down to the mK level, is what would actually move the stability number closer to the theoretical floor from here. If you don't think we're ready, tell me what specific test you'd want to see first.

Where this leaves us

- V17 held closed-loop lock for 88.6 hours straight, no drift — the longest run so far, by a wide margin.
- Best dX stability on record. dY isn't better than V14, and that's worth saying plainly rather than glossing over — the likely cause is the periodic AO unload cycle tracking real thermal drift, not a regression.
- The AO hardware itself is ~30x finer than before, wiring issue fixed and reverified ahead of the run.
- The V11/V12 labeling error is now corrected, sourced against the published thesis report.

Two things I'll be working on next: switching frame-saving to once per 10 minutes (above), and looking at physically damping the external AC ripple. Overall, a good run — and, I think, ready to move to Phase 3.